

Bayesian Inference - rýchly recall:

CIEL: vypočítať posterior $P(\theta | D)$

= čo vieme o parametroch θ po tom, čo sme videli dáta D

BAYES:

$$P(\theta | D) = P(D | \theta) * P(\theta) / \int P(D | \theta) P(\theta) d\theta$$

PROBLÉM:

Menovateľ ($\int \dots$) je intraktabilný pre väčšinu reálnych modelov

→ nedá sa analyticky vypočítať

$$P(\theta | D) = \frac{P(D | \theta)P(\theta)}{\int_{\theta} P(D | \theta)P(\theta)d\theta}$$

RIEŠENIE = MCMC:

Namiesto presného výpočtu použijeme aproximáciu.

MCMC "sampleuje" z posteriuoru bez toho aby ho priamo počítal.

DNEŠNÁ LECTURE:

1. Ako MCMC funguje? (Metropolis, Gibbs, HMC)

2. Ako zistiť či aproximácia je dobrá? (Diagnostics)

MCMC - základná myšlienka:

Namiesto výpočtu posteriuoru priamo → sampleujeme z neho.

Samples aproximujú posterior → môžeme z nich počítať mode, HDI, mean...

Ako? → random walk (náhodná prechádzka po priestore parametrov)

KLÚČOVÁ VEC:

MCMC nepotrebuje menovateľ ($\int P(D|\theta)P(\theta)d\theta$) !

Stačí vedieť vyhodnotiť:

- $P(\theta)$ = prior (vieme, sami ho definujeme)

- $P(D | \theta)$ = likelihood (vieme, sami ho definujeme)

→ To sú presne veci ktoré v PyMC definujeme sami.

→ Menovateľ (intraktabilná časť) sa nepotrebuje!

Metropolis Algorithm:

MYŠLIENKA: random walk v priestore parametrov

→ preferujeme miesta s vyššou posterior hustotou

KROKY:

1. Si na nejakej pozícii θ

2. Proposal distribution navrhne novú pozíciu θ^*

(zvyčajne Gaussian okolo aktuálnej pozície)

3. Prijmeš návrh ak:

- θ^* je lepší (vyššia posterior hustota) → vždy prijmi

- θ^* je horší → prijmi s malou pravdepodobnosťou (náhodná šťastena)

VLASTNOSTI PROPOSAL DISTRIBUTION:

- je fixná počas celého behu algoritmu

- nezávisí od prioru ani likelihood

- je centrovaná na aktuálnej pozícii

PREČO OBČAS PRIJÍMAME AJ HORŠIE NÁVRHY?

→ aby sme neuviazli v lokálnom maxime

→ aby sme preskúmali celý priestor parametrov

ANALOGIA: Kráľ Markov a ostrovy

Kráľ chce navštevovať ostrovy úmerne ich populácii

(= chceme samplovať z posteriuoru úmerne jeho hustote)

OSTROVY = priestor parametrov

POPULÁCIA OSTROVA = posterior hustota v danom bode

KRÁĽOVA NÁVŠTEVA = sample

METROPOLIS PRAVIDLÁ V TEJTO ANALOGII:

- Initial state = na ktorom ostrove kráľ začína

- Proposal dist. = navrhni susedný ostrov (vľavo alebo vpravo)

- Density function = populácia ostrova (= posterior)

- Acceptance ratio = ak má navrhnutý ostrov väčšiu populáciu → choď,
ak menšiu → choď len s pravdepodobnosťou $\text{pop}(\text{nový}) / \text{pop}(\text{aktuálny})$

VÝSLEDOK:

Kráľ trávi čas na ostrovoch úmerne ich popsamplingu z posteriuoru úmerne jeho hustote

PREČO METROPOLIS FUNGUJE?

Kľúčový theorem:

Ak beží dosť dlho → samples konvergujú k posterioru P.

PREČO?

Markov Chain vytvorená Metropolisom je:

- Irreducible: z každého stavu sa dá dostať do každého iného

- Aperiodic: žiadny stav nie je navštevovaný periodicky

Tieto dve vlastnosti garantujú jedinečnú stacionárnu distribúciu = P.

VIZUALIZÁCIA METROPOLIS - výsledky:

PRVÝ GRAF (horizontálny):

X-os = Island ID (1-4), Y-os = time step

→ každá horizontálna čiara = kráľ bol na tomto ostrove v tomto čase

→ ostrov 4 má najviac čiar = najčastejšie navštevovaný = najväčšia populácia

→ ostrov 1 má najmenej čiar = najmenej navštevovaný = najmenšia populácia

DRUHÝ GRAF (az.plot_trace):

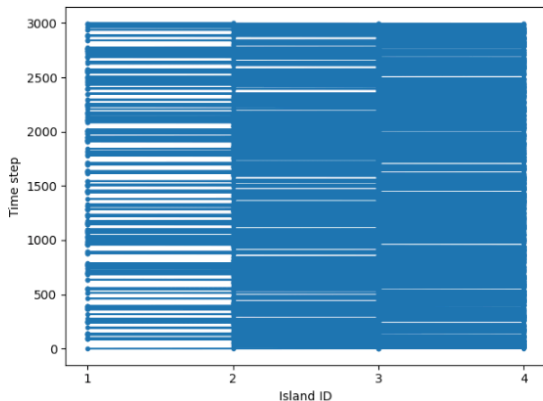
Ľavá strana = histogram návštev každého ostrova

→ stĺpce rastú od 1 do 4 = algoritmus navštevoval ostrovy úmerne ich populácii ✓

Pravá strana = trace plot (časový rad)

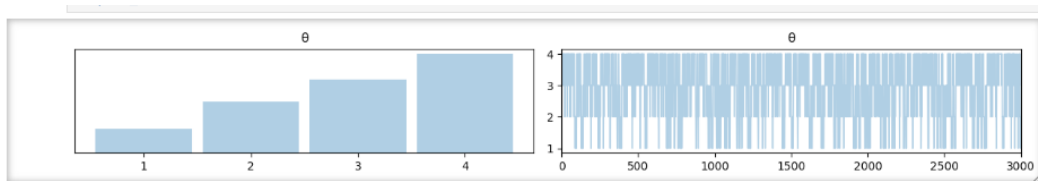
→ vidíme ako kráľ "skákal" medzi ostrovmi v čase

→ náhodne, ale viac času trávil na väčších ostrovoch



ZÁVER:

Metropolis funguje → samples zodpovedajú cieľovej distribúcii P.



METROPOLIS PRE BAYESIAN INFERENCE:

KLÚČOVÁ MYŠLIENKA:

Algoritmus potrebuje len POMER dvoch posteriorov, nie ich absolútne hodnoty.

$$\text{acceptance ratio} = \frac{P(\theta_2 | D)}{P(\theta_1 | D)} = \frac{P(D | \theta_2)p(\theta_2)/Z}{P(D | \theta_1)p(\theta_1)/Z} = \frac{P(D | \theta_2)p(\theta_2)}{P(D | \theta_1)p(\theta_1)}$$

Z sa skrúti! → nepotrebujeme intraktabilný menovateľ!

TEDA STAČÍ VEDIEŤ:

- $P(D|\theta)$ = likelihood → sami definujeme

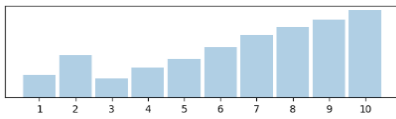
- $P(\theta)$ = prior → sami definujeme

SPOJENIE S KRÁĽOVÝM PRÍKLADOM:

Rovnaký algoritmus, len:

- "populácia ostrova" = likelihood * prior

- výber proposal distribution ovplyvňuje rýchlosť konvergencie

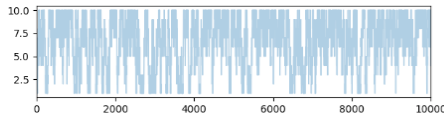


VÝSLEDOK - Metropolis pre Bayesian Inference:

ĽAVÝ GRAF (histogram):

Ostrov 1-10, stĺpce rastú zľava doprava.

- algoritmus navštevoval ostrovy úmerne ich posterior hustote ✓
- ostrov 10 = najvyšší posterior → najčastejšie navštevovaný



PRAVÝ GRAF (trace plot):

- 10 000 viac času trávi na vyšších ostrovochhodne.
- ale viac času trávi na vyšších ostrovoch (7-10)
- vyzerá "chaoticky" ale to je správne - dobrý mixing

ZÁVER:

Metropolis správne aproximuje posterior
aj keď sme nikdy nepočítali Z (menovateľ).

Beta-Bernoulli model:

NOVÝ MODEL na demonštráciu Metropolis:

$y \sim \text{Bernoulli}(p = \theta)$ → y = výsledky hodu mincou (0 alebo 1)

$\theta \sim \text{Beta}(\alpha=1, \beta=1)$ → prior na pravdepodobnosť (= uniform, nič nevieme)

θ = pravdepodobnosť že padne hlava

y = vektor N hodov mincou

PREČO TENTO MODEL?

→ jednoduchý, dobre pochopiteľný

→ použijeme ho na ukázanie **PROBLÉMOV** Metropolis algoritmu

Beta distribúcia:

ČO JE TO?

Distribúcia pre parametre ktoré sú medzi 0 a 1

→ ideálna pre pravdepodobnosti (napr. $\theta = P(\text{hlava})$)

PREČO JE UŽITOČNÁ?

Je conjugate prior pre Bernoulli/Binomial.

→ ak prior = Beta(α, β) a dáta sú coin tosses,

posterior je tiež Beta:

$P(\theta | \text{dáta}) = \text{Beta}(\alpha + z, \beta + N - z)$

kde z = počet 1-čiek, N = celkový počet hodov

→ analytické riešenie! (nepotrebujeme MCMC pre tento model)

→ ale použijeme ho na testovanie MCMC

GRAFY - intuícia pre α a β :

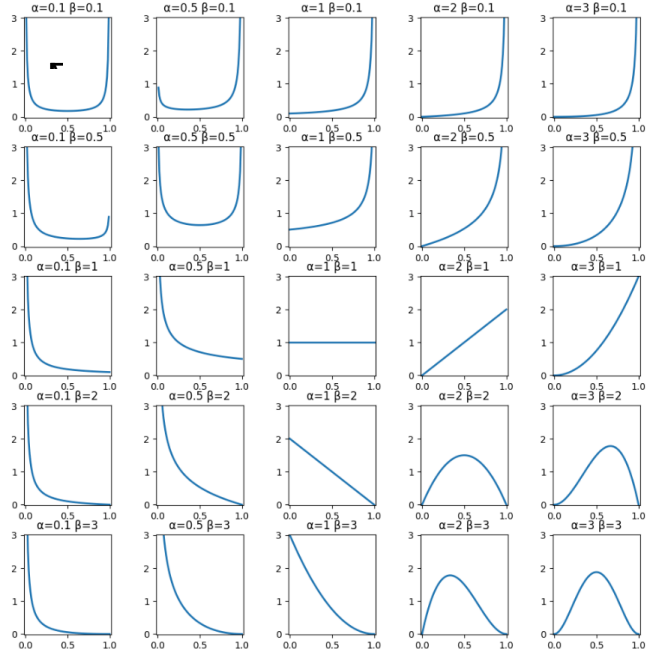
$\alpha=1, \beta=1$ → uniform (nič nevieme)

$\alpha > \beta$ → θ bližšie k 1 (viac hláv)

$\alpha < \beta$ → θ bližšie k 0 (viac písem)

$\alpha = \beta > 1$ → symetrický kopec v strede

malé α, β → hmota na krajoch (buď takmer 0 alebo takmer 1)



MODEL V PYMC:

PREČO TO ROBIEME?

Chceme ukázať PROBLÉMY Metropolis algoritmu.

Beta-Bernoulli je ideálny príklad lebo:

- má analytické riešenie (vieme presný posterior)
- môžeme porovnať: "čo by malo byť" vs "čo Metropolis dostane"
- ak Metropolis zlyhá, hneď to vidíme

DÁTA:

20 hodov mincou: 14x hlava (1) + 6x písmo (0)

MODEL:

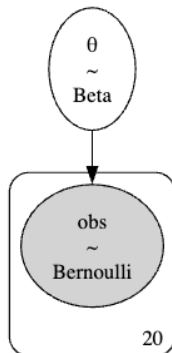
$\theta \sim \text{Beta}(\alpha=1, \beta=1) \rightarrow$ uniform prior (nič nevieme o minci)

$y \sim \text{Bernoulli}(p=\theta) \rightarrow$ každý hod závisí od θ

OČAKÁVANÝ POSTERIOR (analyticky):

$\text{Beta}(15, 7) \rightarrow \theta$ okolo 0.7 (14/20 hláv)

Teraz uvidíme či Metropolis dostane rovnaký výsledok.



EFEKT VARIANCE NA PROPOSAL DISTRIBUTION:

OTÁZKA: Ako veľké kroky má Metropolis robiť?

Testujeme 3 hodnoty σ : 0.02, 0.2, 2.0

$\sigma=0.02$ (príliš malé kroky):

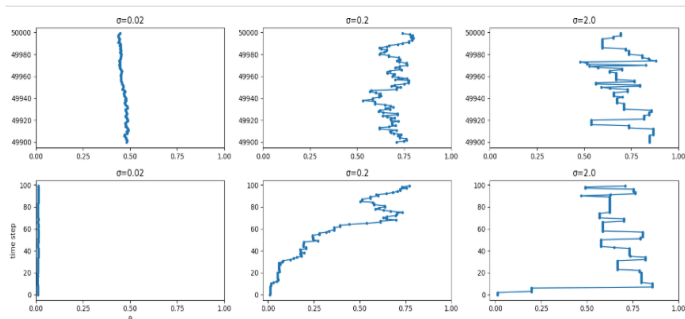
- chain sa pomaly plazí, nepreskúma celý priestor
- histogram má divný tvar (nie pekná krivka)
- zlý mixing

$\sigma=0.2$ (správne):

- chain skáče dobre, preskúma celý priestor
- histogram vyzerá ako $\text{Beta}(15,7)$ ✓
- dobrý mixing

$\sigma=2.0$ (príliš veľké kroky):

- chain skáče príliš ďaleko, veľa návrhov sa odmieta
- výsledok je ok, ale menej efektívny



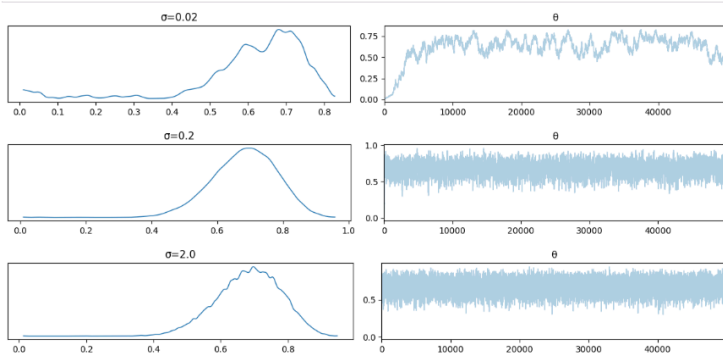
BURN-IN / TUNE:

Na začiatku chain štartuje zo zlého miesta ($\theta=0.01$)

- prvých N samples sú "odpad" (burn-in)
- preto nastavujeme tune > 0 (zvyčajne 500-1000)
- tieto samples sa zahadzujú

ZÁVER:

Výber σ v proposal distribution je kritický.
Príliš malé = pomalé, príliš veľké = neefektívne.
→ Toto je hlavný problém Metropolis algoritmu.



KORELOVANÉ SAMPLES - ďalší problém Metropolis:

PROBLÉM:

Každá motivácia pre lepšie liezko* predchádzajúceho

→ susedné samples sú korelované

→ nepridávajú veľa novej informácie o posteriore

AUTOCORRELATION:

Meria koreláciu medzi $\text{sample}[t]$ a $\text{sample}[t+k]$

k = lag (počet krokov vpred)

Ideál: rýchly pokles k 0 → samples sú nezávislé

GRAFY (autocorrelation pre lag 0-100):

$\sigma=0.02$: vysoká korelácia aj pri lag=100

→ samples sú takmer identické so susedmi

→ veľmi zlý mixing, potrebuješ obrovské množstvo samples

$\sigma=0.2$: rýchly pokles k 0

→ samples sú relatívne nezávislé ✓

→ dobrý mixing

$\sigma=2.0$: okamžitý pokles k 0

→ samples sú nezávislé, ale veľa sa odmieta

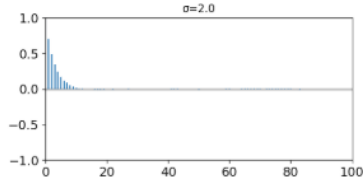
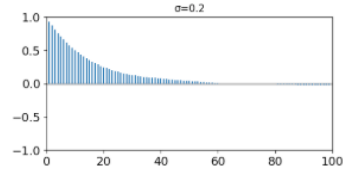
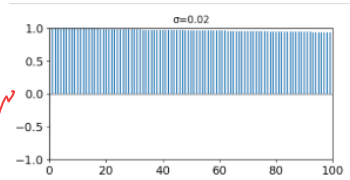
→ efektívne ale plytvá výpočtami

ZÁVER:

Metropolis trpí koreláciou samples.

Čím menšie σ , tým horšie.

→ Motivácia pre lepšie algoritmy (Gibbs, HMC)



EFFECTIVE SAMPLE SIZE (ESS):

ČO JE TO?

Hovorí koľko SKUTOČNE nezávislých samples chain obsahuje.

→ lebo korelované samples = menej informácie

VZOREC:
$$ESS = \frac{N}{1 + 2 \sum_{k=1}^{\infty} \text{ACK}(k)}$$

→ čím vyššia autokorelácia, tým nižší ESS

IDEÁL: $ESS \approx N$ (každý sample pridáva novú informáciu)

VÝSLEDKY ($N = 50\,000$ samples):

$\sigma=0.02$: $ESS = 18$ → z 50 000 samples len 18 užitočných!

$\sigma=0.2$: $ESS = 1\,728$ → lepšie, ale stále len 3.5%

$\sigma=2.0$: $ESS = 9\,053$ → najlepšie, ~18% užitočných

$ESS (\sigma=0.02)$: 17.55079934142351 out of 50000

$ESS (\sigma=0.2)$: 1728.166453266916 out of 50000

$ESS (\sigma=2)$: 9053.302915553377 out of 50000

ZÁVER:

Metropolis s malým σ je extrémne neefektívny.

50 000 samples ale len 18 užitočných = obrovské plytvanie.

→ PyMC varuje keď $ESS < 25\%$ z N

→ Motivácia pre HMC ktorý má oveľa vyšší ESS

Gibbs Algorithm:

ČO JE TO?

Modifikácia Metropolis kde návrhy sú VŽDY prijaté.

→ žiadne odmietanie → efektívnejší

AKO FUNGUJE?

Pre posterior s viacerými parametrami ($\theta_1, \theta_2, \dots, \theta_n$):

→ samplejme každý parameter ZVLÁŠŤ, ostatné drží fixné

→ $P(\theta_1 | \theta_2, \theta_3, \dots, D)$, potom $P(\theta_2 | \theta_1, \theta_3, \dots, D)$, atď.

PREČO SÚ NÁVRHY VŽDY PRIJATÉ?

Lebo samplejeme priamo z marginálneho posterioru

→ každý sample je "lokálne" správny

OBMEDZENIE:

Funguje LEN s conjugate priors.

Conjugate prior = prior a posterior sú z rovnakej rodiny distribúcií.

Príklad: Beta prior + Bernoulli likelihood → Beta posterior ✓

Väčšina reálnych modelov conjugate priors nemá

→ preto Gibbs dnes ustupuje HMC.

Gibbs funguje len vtedy keď pre každý parameter vieš rýchlo vypočítať jeho posterior priamo vzorcom. Ak to nevieš, Gibbs nefunguje. Hotovo.



2D Beta-Bernoulli - Gibbs v praxi:

SITUÁCIA:

Dve nezávislé mince, každá má svoju vlastnú pravdepodobnosť.

θ_1 = bias prvej mince

θ_2 = bias druhej mince

→ 2 parametre = 2 dimenzie = ideálne pre Gibbs

GIBBS POSTUP:

1. Fixuj θ_2 , samplej θ_1 z $\text{Beta}(z_1+1, N_1-z_1+1)$

2. Fixuj θ_1 , samplej θ_2 z $\text{Beta}(z_2+1, N_2-z_2+1)$

3. Opakuj

Funguje lebo Beta je conjugate prior pre Bernoulli ✓

VÝSLEDKY (grafy):

θ_1 → posterior okolo 0.7 (14/20 hláv)

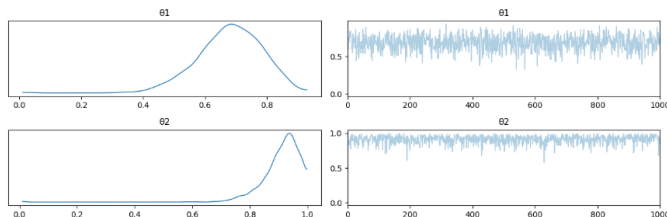
θ_2 → posterior okolo 0.85 (vyššia bias)

Trace ploty vyzerajú dobre → rýchly mixing ✓

ZÁVER:

Gibbs funguje skvele pre tento príklad.

Ale len preto, že máme conjugate priors.



ĽAVÁ STRANA (histogram = posterior):

θ_1 : vrchol okolo 0.7 → minca 1 padá ~70% hlavou

θ_2 : vrchol okolo 0.85 → minca 2 padá ~85% hlavou

PRÁVÁ STRANA (trace plot = priebeh samplingu):

θ_1 : skáče rýchlo medzi 0.4-0.8 → dobrý mixing ✓

θ_2 : skáče rýchlo okolo 0.85 → dobrý mixing ✓

METROPOLIS vs GIBBS - jednoducho:

Predstav si že hľadáš vrchol hory v hmle.

METROPOLIS:

Urobíš náhodný krok ľubovoľným smerom.

Ak si vyššie → zostaneš tam.

Ak si nižšie → možno sa vrátiš, možno nie.

Problém: kroky môžu byť príliš malé alebo príliš veľké.

GIBBS:

Namiesto náhodného kroku sa pohybuješ len

v JEDNOM smere naraz (napr. len sever-juh).

Vždy nájdeš najlepšie miesto v tom smere.

Potom sa pohybuješ len východ-západ.

A tak striedaš.

Výhoda: každý krok je vždy prijatý (nikdy sa nevraciaš).

PREČO "VIACERO PARAMETROV"?

Každý parameter = jeden smer v priestore.

Napr. model má α a β → 2 dimenzie → 2 smery.

Gibbs ich rieši jeden po druhom.

OBMEDZENIE GIBBS:

Funguje len ak vieš analyticky vypočítať

"najlepšie miesto v danom smere" → conjugate priors.

Ak nie → musíš použiť Metropolis alebo HMC.

Máme dve mince a chceme zistiť bias každej z nich.

MINCA 1: hodili sme 20x, padlo 14x hlava → $\theta_1 \approx 0.7$

MINCA 2: hodili sme 20x, padlo 17x hlava → $\theta_2 \approx 0.85$

GIBBS HOVORÍ:

"Neviem naraz odhadnúť obe, tak ich budem striedať"

Krok 1: Predstieraj že θ_2 už vieš → odhadni θ_1

Krok 2: Predstieraj že θ_1 už vieš → odhadni θ_2

Krok 3: Opakuj 1000x

Po 1000 opakovaniach máš celkom dobrý odhad oboch.

PREČO TO FUNGUJE?

Lebo pre každý krok vieš analyticky vypočítať

"aký je najlepší odhad θ_1 keď θ_2 je fixný?"

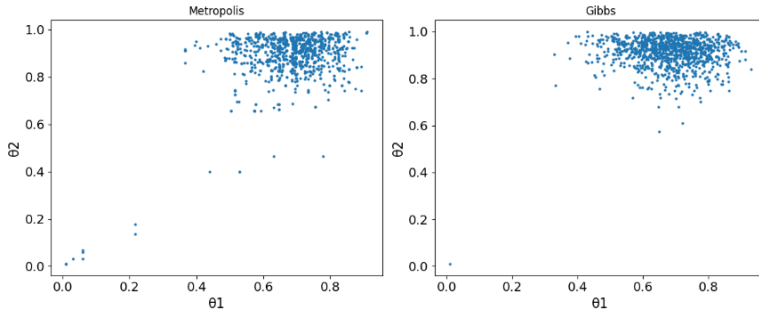
→ to je conjugate prior v praxi

→ jednoducho dosadiš do vzorca $\text{Beta}(\text{počet hláv}+1, \text{počet opakov}+1)$

Gibbs fungoval dobre → rýchlo našiel správne hodnoty
bez burn-in problémov ktoré mal Metropolis.

METROPOLIS vs GIBBS - porovnanie:

Rovnaký model, rovnaký počet samples (1000), rovnaký štart ($\theta_1 = \theta_2 = 0.01$)



METROPOLIS (ľavý graf):

- Vidiš body dole vľavo (blízko 0,0) → burn-in
- Chain sa pomaly "plazi" smerom k správnej oblasti (0.7, 0.85)
- Veľa samples je odpad kým sa dostane na správne miesto

GIBBS (prvý graf):

- Takmer žiadne body mimo správnej oblasti
- Hneď od začiatku sampleuje v správnej oblasti
- Žiadny burn-in problém

ZÁVER:

Gibbs je oveľa efektívnejší pre tento príklad. 1000 samples Gibbs > 1000 samples Metropolis lebo Metropolis plytvá samples počas burn-in.

ESS = Effective Sample Size = koľko samples je skutočne užitočných.

```
Metropolis
<xarray.Dataset> Size: 168
Dimensions: ()
Data variables:
  theta1 float64 8B 167.3
  theta2 float64 8B 134.6
Attributes:
  created_at: 2026-03-17T18:20:53.246590+00:00
  arviz_version: 0.23.4
  inference_library: pymc
  inference_library_version: 5.28.0
  sampling_time: 0.0813291072845459
  tuning_steps: 0
```

ESS porovnanie (z 1000 samples):

Metropolis:

theta1: ESS = 167 → z 1000 samples len 167 užitočných (17%)
theta2: ESS = 135 → z 1000 samples len 135 užitočných (14%)

Gibbs:

theta1: ESS = 958 → z 1000 samples 958 užitočných (96%)!
theta2: ESS = 778 → z 1000 samples 778 užitočných (78%)!

```
Gibbs
<xarray.Dataset> Size: 168
Dimensions: ()
Data variables:
  theta1 float64 8B 958.2
  theta2 float64 8B 777.5
```

ZÁVER:

Gibbs je dramaticky lepší pre tento prípad.
Metropolis plytvá ~85% samples.
Gibbs využíva takmer každý sample.

LIMITY METROPOLIS A GIBBS:

PROBLÉM: vysoké dimenzie (veľa parametrov)
Jav sa volá "concentration of measure"

JEDNODUCHÁ INTUÍCIA:

1D: pravdepodobnosť je sústredená okolo stredu (módu)
→ Metropolis/Gibbs ľahko nájdu správnu oblasť

Ale vo vysokých dimenziách:

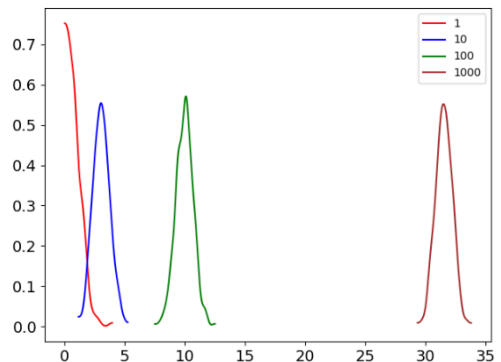
- väčšina pravdepodobnosti je ĎALEKO od módu
- v "škrupe" okolo stredu, nie v strede samotnom
- Metropolis/Gibbs hľadajú okolo módu → minú väčšinu posterioru

GRAF (vzdialenosť náhodného bodu od stredu):

1 dimenzia → vzdialenosť ≈ 0-1
10 dimenzií → vzdialenosť ≈ 3
100 dimenzií → vzdialenosť ≈ 10
1000 dimenzií → vzdialenosť ≈ 31

Čím viac parametrov, tým ďalej je "dôležitá oblasť" od stredu.
Metropolis a Gibbs to nevedia efektívne preskúmať.

→ MOTIVÁCIA PRE HMC ktorý toto rieši.



X-os = vzdialenosť náhodného bodu od stredu (módu)
Y-os = pravdepodobnosť tejto vzdialenosti

Hamiltonian Monte Carlo (HMC):

HLAVNÁ MYŠLIENKA:

Metropolis skáče náhodne → nevie kam ísť.
HMC používa gradient posterioru → vie kam ísť.

ANALOGIA:

Metropolis = opitý človek ktorý blúdi v tme
HMC = lyžiar ktorý cíti sklon svahu a ide po ňom

AKO FUNGUJE?

Používa fyzikálnu simuláciu (ako guľička na kopci):
→ cíti sklon (gradient) posterioru
→ nasleduje strmý gradient = rýchlejšie nájde správnu oblasť
→ návrhy sú oveľa lepšie ako náhodné

KOMPROMIS:

HMC je drahší na výpočet (treba počítať gradient)
ale potrebuje oveľa menej samples
→ celkovo rýchlejší a efektívnejší

→ Toto je algoritmus ktorý PyMC používa (NUTS = varianta HMC)

Hamiltonian - fyzikálny základ HMC:

ZÁKLADNÁ MYŠLIENKA Z FYZIKY:

Celková energia systému = potenciálna + kinetická energia
 $H(\theta, \phi) = E(\theta) + K(\phi)$

V NAŠOM KONTEXTE:

θ = pozícia (= parametre ktoré hľadáme)
 ϕ = momentum (= náhodný "kopanec" ktorý dostaneme)
 $E(\theta)$ = negative log posterior (= "kopec/údolie")
 $K(\phi)$ = kinetická energia (= rýchlosť guľičky)

KLÚČOVÁ VLASTNOSŤ:

Celková energia H je konštantná počas simulácie.
→ ak guľička ide dolu (nižší E) → zrýchli (vyšší K)
→ ak guľička ide hore (vyšší E) → spomalí (nižší K)

PREČO JE TO UŽITOČNÉ?

Oblasť s nízkou potenciálnou energiou = vysoká posterior hustota
→ guľička tu ide pomalšie → viac samples v správnych oblastiach ✓
Oblasť s vysokou potenciálnou energiou = nízka posterior hustota
→ guľička tu ide rýchlo → málo samples v nepotrebných oblastiach ✓

HMC algoritmus - ako funguje v praxi:

KROKY (striedame):

1. Vyber náhodné momentum $p \sim \text{Normal}(0,1)$
2. Simuluj fyziku pomocou Leapfrog metódy:
 - pol krok v p (momentum)
 - celý krok v x (pozícia)
 - pol krok v p (momentum)
 - opakuj Tau krát
3. Porovnaj energiu H pred a po simulácii:
 - $dH < 0$ (nižšia energia) → vždy prijmi
 - $dH > 0$ → prijmi s pravdepodobnosťou $e^{-(dH)}$
 - ak H sa veľmi zmenilo → numerická chyba → odmietni

PREČO LEAPFROG?

Dva pol-kroky namiesto jedného celého = väčšia presnosť
(výsledok z teórie numerickej integrácie)

NA KONCI:

Ignorujeme momentum p , zachováme len pozíciu $x = \theta$
→ to sú naše samples z posterioru

HMC - intuícia cez fyziku:

ANALOGIA: guľička na kopci

Posterior = kopec (vysoká hustota = vrchol)
HMC obráti kopec → údolie (negative log posterior = "potential")
Guľička sa kotúľa dole do údolia = smeruje k vysokej posterior hustote

KROKY:

1. Si na nejakej pozícii (current position)
2. Dostaneš náhodný "kopanec" (random momentum)
3. Guľičke necháš voľne sa kotúľať podľa fyziky
4. Kde pristane = nový návrh

PREČO JE TO LEPŠIE?

→ guľička CÍTI tvar posterioru (gradient)
→ prirodzene smeruje do oblastí s vysokou hustotou
→ návrhy nie sú náhodné, sú "inteligentné"

PREČO PYTENSOR/TENSORFLOW?

Fyzikálna simulácia vyžaduje gradient posterioru.
→ treba automatické derivovanie (differentiable)
→ preto PyMC používa Pytensor pod kapotou

VÝSLEDOK (bottom row):

Proposals nie sú symetrické okolo aktuálnej pozície.
→ viac proposals tam kde je vysoká hustota
→ oveľa vyšší ESS ako Metropolis/Gibbs

HMC - formálna definícia (zjednodušené):

KLÚČOVÝ VZŤAH:

$P(\theta) = e^{-(E(\theta))} / Z$
→ nízka energia E = vysoká pravdepodobnosť P
→ teda hľadáme miesta s nízkou energiou = vysokým posteriorom

PREČO PYMC POUŽÍVA LOG-PRAVDEPODOBNOSŤ?

$P(x) = e^{-(\log p(x))}$
→ derivácia e^x je e^x → ľahko počítateľná
→ Pytensor to vie automaticky derivovať (gradient)
→ bez gradientu HMC nefunguje

PRE KAŽDÝ PARAMETER θ_j :

HMC pridá momentum ϕ_j
→ sample = dvojica (θ, ϕ)
→ θ = kde sme (posterior hustota)
→ ϕ = ako rýchlo sa pohybuje (kinetická energia)

VELKOSŤ SKOKU:

Väčší momentum ϕ → väčší skok v θ
→ **HMC si sám reguluje veľkosť krokov**
→ preto nepotrebuje ručne nastavovať σ ako Metropolis

HMC - dôležité details (jednoducho):

SPOJITÝ PARAMETER:

= parameter ktorý môže byť akékoľvek číslo (0.1, 0.537, 3.14...)
Príklad: θ = pravdepodobnosť hlavy pri minci → môže byť čokoľvek medzi 0 a 1
Opak = kategorický parameter (napr. kontinent: Afrika alebo nie)
HMC funguje len pre spojité → preto PyMC niekedy kombinuje algoritmy

ACCEPTANCE RATE:

= koľko percent návrhov algoritmus prijme

Metropolis s malým α : acceptance rate ~5%

→ z 1000 návrhov prijme len 50 → veľké plytvanie

HMC: acceptance rate ~90%+

→ z 1000 návrhov prijme 900 → efektívne

PREČO HMC NIEKEDY ODMIETNE?

Fyzikálna simulácia nie je 100% presná.
Keď energia "skočí" príliš → numerická chyba → odmietni.
Ale stáva sa to zriedka.

ZÁVER:

HMC robí múdrejšie návrhy (fyzika + gradient)
→ takmer vždy trafi správnu oblasť
→ takmer vždy prijme → menej odpadu → vyšší ESS

HMC - parametre a NUTS:

DVA HLAVNÉ PARAMETRE:

Epsilon (ϵ) = veľkosť kroku simulácie

→ malý ϵ = presnejší ale pomalší

→ veľký ϵ = rýchlejší ale môže "preskočiť" správnu oblasť

Steps (τ) = počet krokov simulácie

→ málo krokov = pohybujeme sa pomaly

→ veľa krokov = môžeme "prestreliť" a ísť späť (U-turn)

PRAVÝ STĹPEC V GRAFE = problém:

Príliš veľa krokov → guľička urobí U-turn

→ skončí blízko kde začala → plytvanie

WARM-UP (tune v PyMC):

HMC si automaticky nastaví ϵ a τ počas warm-up fázy

Cieľ = dosiahnuť target acceptance rate

→ kniha odporúča 65%, PyMC default = 95%

Warm-up v HMC je iný ako burn-in v Metropolis:

→ nie je to odpad, je to ladenie parametrov

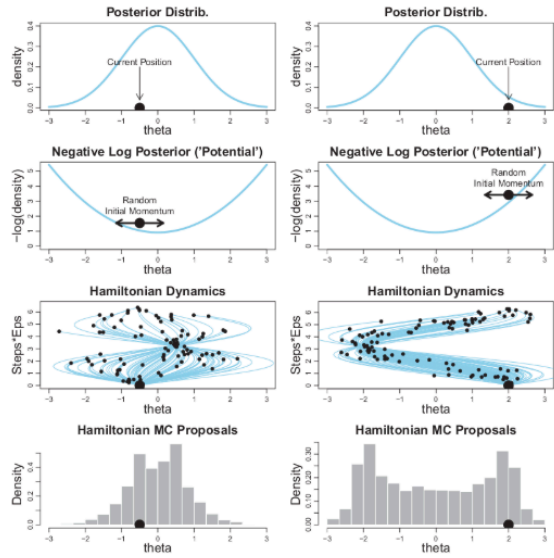
NUTS (No U-Turn Sampler):

= vylepšená verzia HMC ktorú PyMC používa defaultne

→ automaticky detekuje U-turn a zastaví simuláciu

→ nepotrebuje ručne nastavovať τ

→ efektívnejší ako klasický HMC



HMC - detekcia chýb:

VÝHODA HMC OPROTI METROPOLIS/GIBBS:

HMC vie detekovať keď niečo ide zle.

Metropolis a Gibbs zlyhávajú potichu - ty o tom nevieš.

HMC ti povie keď je problém.

DVA HLAVNÉ VAROVANIA V PYMC:

1. Target accept not reached:

Počas warm-up sa nepodarilo dosiahnuť cieľovú acceptance rate.

→ model má pravdepodobne problém

2. Divergences:

Simulácia "vyletela" preč od posterioru.

= energia skočila príliš ďaleko od predchádzajúceho stavu.

→ často sa stáva pri posterioroch s ostrými krivkami

→ PyMC ti povie koľko divergencií nastalo

FOLK THEOREM (Andrew Gelman):

"Keď máš výpočtové problémy,

často je problém s tvojím modelom."

→ Divergencie nie sú len technický problém.

→ Sú signál že model je zle špecifikovaný.

VIZUÁLNA DIAGNOSTIKA - trace plot:

ČO HĽADÁME?

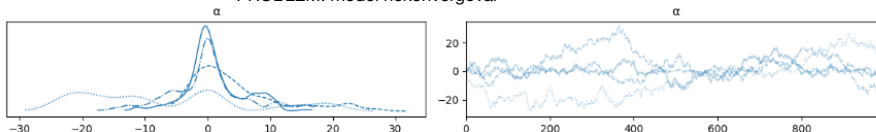
1. Histogram: všetky chains (čiary) sa prekrývajú → dobrá konvergencia
2. Trace plot: chain rýchlo skáče hore-dole → dobrý mixing

HORNÉ GRAFY (zlý model - divergencie):

Histogram: chains sa neprekrývajú → každá chain našla iný posterior!

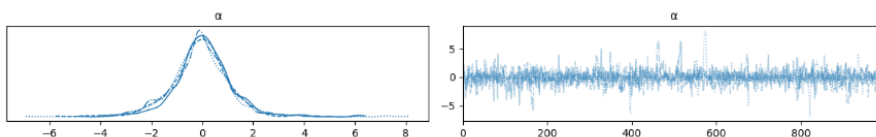
Trace plot: chains sa pohybujú pomaly, plazia sa → zlý mixing

→ PROBLÉM: model nekonvergoval



- On the contrary, the trace below shows great overlap of all chains and very good mixing or the right-hand-side

```
pm.plot_trace(trace_fixed, var_names=['alpha']);
```



DOLNÉ GRAFY (dobrý model):

Histogram: všetky chains sú takmer identické → konvergencia ✓

Trace plot: rýchle náhodné skákanie → dobrý mixing ✓

→ môžeme veriť výsledkom

PRAVIDLO PALCA:

Dobrý trace plot = "chlpatá húsenica"

→ hustý, rýchly, náhodný

Zlý trace plot = "had ktorý sa plazí"

→ pomalý, trendový, chains sa rozchádzajú

MCMC DIAGNOSTIKA - ESS, MCSE a R-hat:

1. ESS (Effective Sample Size) - už vieme:

Koľko samples je skutočne užitočných.

Odporúčanie: aspoň 500 efektívnych samples.

2. MCSE (Monte Carlo Standard Error):

$MCSE = SD / \sqrt{ESS}$

= chyba nášho odhadu parametra

Dobrý model: MCSE blízko 0.0

Divergentný model: $mcse_mean = 3.6$ (obrovská chyba!)

Dobrý model: $mcse_mean = 0.035$ (takmer nula) ✓

```
5): pm.summary(trace_mh_divergent)
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
α	0.141	9.878	-22.450	19.307	3.628	2.582	9.0	13.0	1.43
σ	35.789	89.206	0.506	124.437	8.086	14.961	19.0	365.0	1.15

3. R-hat:

Meria či všetky chains konvergovali k rovnakému posterioru.

= pomer variability MEDZI chains vs VNÚTRI chains

R-hat = 1.0 → chains sa zhodujú → konvergencia ✓

R-hat > 1.0 → chains sa rozchádzajú → problém!

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
α	-0.037	1.201	-2.318	2.064	0.035	0.053	1378.0	1374.0	1.0
σ	1.548	0.813	0.548	3.072	0.023	0.029	1254.0	1595.0	1.0

VÝSLEDKY:

Divergentný model: R-hat = 1.43 (α), 1.15 (σ) → ZĽÉ

Dobrý model: R-hat = 1.003 (α), 1.004 (σ) → DOBRÉ ✓

PRAVIDLO:

R-hat < 1.01 → ok

R-hat > 1.01 → problém s modelom

```
[27]: pm.rhat(trace_mh_divergent).items()
```

```
[27]: ItemsView(<xarray.Dataset> Size: 168
```

```
Dimensions: ()
```

```
Data variables:
```

```
α float64 8B 1.426
```

```
σ float64 8B 1.145
```

```
Attributes:
```

```
created_at: 2026-03-17T18:20:59.728908+00:00
```

```
arviz_version: 0.23.4
```

```
inference_library: pymc
```

```
inference_library_version: 5.28.0
```

```
sampling_time: 0.18256211280022754
```

```
tuning_steps: 0)
```

```
[28]: pm.rhat(trace_fixed).items()
```

```
[28]: ItemsView(<xarray.Dataset> Size: 168
```

```
Dimensions: ()
```

```
Data variables:
```

```
α float64 8B 1.003
```

```
σ float64 8B 1.004
```

```
Attributes:
```

```
created_at: 2026-03-17T18:21:01.629683+00:00
```

```
arviz_version: 0.23.4
```

```
inference_library: pymc
```

```
inference_library_version: 5.28.0
```

```
sampling_time: 0.4809541702270508
```

```
tuning_steps: 1000)
```

R-HAT POROVNANIE:

Divergentný model (tuning_steps = 0):

α: R-hat = 1.426 → chains sa veľmi rozchádzajú → ZLÉ

σ: R-hat = 1.145 → tiež zlé

Dobrý model (tuning_steps = 1000):

α: R-hat = 1.003 → chains takmer identické → DOBRÉ ✓

σ: R-hat = 1.004 → takmer identické → DOBRÉ ✓

KLÚČOVÝ ROZDIEL:

Divergentný model mal tune = 0

→ HMC nemal čas nastaviť si parametre (ϵ , τ)

→ výsledok bol katastrofálny

Dobrý model mal tune = 1000

→ HMC si správne nastavil parametre počas warm-up

→ chains konvergovali ✓

ZÁVER:

Vždy nastav tune na aspoň 500-1000!